

Mini Hiring Pipeline — Architecture Summary

GitHub Repository: https://github.com/Deekshithpatkar/Hiring_Pipeline

Candidate: Deekshith Patkar | **Role:** Technical Take-Home Assignment

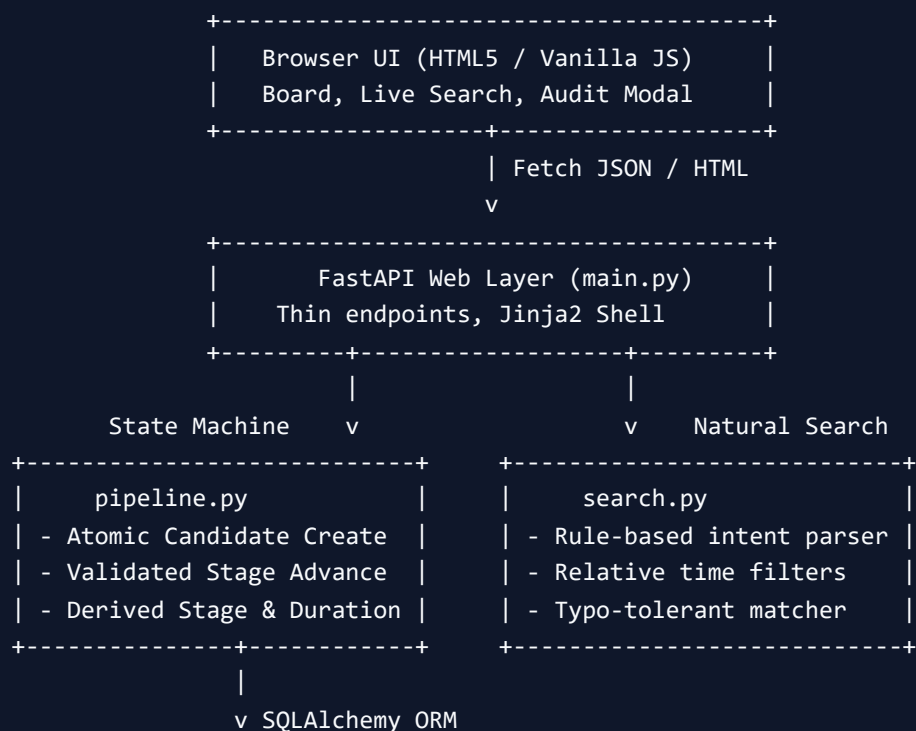
Stack: Python 3.14, FastAPI, SQLAlchemy ORM, PostgreSQL 18, Jinja2, Vanilla JS / CSS

1. System Overview & Core Principles

The Mini Hiring Pipeline is built to manage candidates for a job through a linear stage sequence with three non-negotiable architectural guarantees:

- **Single Source of Truth:** There is no mutable `current_stage` column anywhere. A candidate's current stage is always dynamically derived as the `to_stage` of their latest recorded `StageEvent`. State drift is impossible by design.
- **Database-Level Immutability:** The audit trail is strictly append-only. A PostgreSQL `BEFORE UPDATE OR DELETE` trigger raises an exception directly inside the database engine if any modification or deletion is attempted.
- **Deterministic, Explainable Logic:** State machine validation and search query parsing are written in plain, transparent Python without relying on external NLP libraries or non-deterministic LLM calls.

2. Architecture Diagram



```
+-----+
|               PostgreSQL Database               |
| 1. candidates (id, name, email, created_at)      |
| 2. stage_events (id, candidate_id, from, to, timestamp) |
| * Immutable: Enforced by Postgres BEFORE trigger  |
| * Unique: (candidate_id, to_stage) constraint     |
+-----+
```

3. Data Model & Integrity Guarantees

The database schema consists strictly of two tables:

- `candidates` : Stores the candidate identity (`id`: UUID , `name`: TEXT , `email`: TEXT , `created_at`: TIMESTAMPTZ).
- `stage_events` : The append-only audit trail (`id`: UUID , `candidate_id`: UUID FK , `from_stage`: TEXT NULL , `to_stage`: TEXT , `timestamp`: TIMESTAMPTZ).

PostgreSQL Immutability Trigger

```
CREATE OR REPLACE FUNCTION prevent_stage_events_modification()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'stage_events audit log is immutable and cannot be updated or deleted';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_stage_events_immutable
BEFORE UPDATE OR DELETE ON stage_events
FOR EACH ROW
EXECUTE FUNCTION prevent_stage_events_modification();
```

Preventing Duplicate Stage Events

A table-level constraint `uq_candidate_to_stage (candidate_id, to_stage)` guarantees at the database level that no candidate can ever re-enter a stage they have previously occupied.

4. State Machine Implementation

The pipeline transitions candidates along a strict, forward-only path:

Applied → Screening → Interview → Offer → Hired

- **Atomic Creation:** Creating a candidate writes the `Candidate` row and the initial `from_stage: NULL, to_stage: 'Applied'` event in one database transaction with automatic rollback on error.
- **No Skipping or Reversals:** Candidates can advance only one stage forward at a time. Backward moves (e.g. `Interview → Screening`) or skipped stages (e.g. `Applied → Interview`) are rejected with clear, explanatory error strings.
- **Rejection Handling:** `Rejected` is reachable from any stage before `Hired`. Both `Hired` and `Rejected` are terminal states.
- **Inline Error Feedback:** All validation errors return structured HTTP 400 responses with exact descriptions, enabling immediate inline feedback on candidate cards without confirmation dialogs.

5. Search Engine & Ranking

The search engine uses a rule-based parser in pure Python (no LLM, no external NLP dependencies):

- **Fuzzy Typo-Tolerant Matching:** Employs `difflib.SequenceMatcher` with token analysis at a 0.75 threshold (e.g. "sharam" → "Sharma"). Exact matches rank above fuzzy matches.
- **Relative Time & Transition Filters:** Resolves phrases such as "stuck in Screening for more than a week" (> 7 days) and "moved to Interview since Monday" into UTC timestamp comparisons.
- **Reached-but-not-current:** Identifies queries like "reached Offer but didn't get hired" by verifying historical `offer` events while ensuring the current stage is not `Hired`.
- **Negation:** Parses "everyone except rejected" into an exclusion filter (`current_stage != 'Rejected'`).
- **Diagnostic Guidance:** Unrecognized queries (e.g. "who's in Onboarding") return explicit explanations of what was invalid rather than silent empty lists.